

作业 1: GPU 与 GPU 编程

Weiming HPC Training Camp × LCPU AI Infra Seminars · Session 1

Preface

此次作业为 Session 1 的配套练习，内容覆盖 CUDA Programming Guide¹ 第一、第二部分的全部章节。

共有七种题型：CONCEPT 概念题、FILL-IN 填空题、MODIFY 改造题、DEBUG 修 bug 题、EXPERIMENT 实验题、HANDS-ON 动手题、FROM-SCRATCH “从零实现”题。标 Optional 的为选做题。涉及代码编辑的题目会在标题行右侧标出相应代码文件的路径（相对 assignment01/）。FROM-SCRATCH 题会配套判测脚本。

代码在仓库 assignment01/ 目录，环境配置见其中的 README.md。

AI Policy: CLAUDE.md 或 AGENTS.md。核心原则是“AI 可以帮你理解，但不能替你实现”。

个人实验记录（整理至 2026-09-11）

本稿保留原题，并按题号录入已提供的终端测量和已确认的学习记录；未填写处仍待完成，不代表整份作业已完成。记录中的 PASS 来自远程终端输出，未在本地重新运行 CUDA。模块 0、1 的硬件记录为 RTX 3080 Ti (compute capability 8.6)；模块 7 的 softmax benchmark 再次打印确认 GPU 为 RTX 3080 Ti。模块 2.3 的两组重复测量来自同一新容器，当时未记录的 GPU 型号仍不作追溯推定。现已补充模块 7 的讨论、测试与性能数据，以及模块 8 的作答、PTX 观察和编译兼容性实验。

Content

Module	主题	对应阅读	代码位置
0	环境准备	README.md	cuda/m0_env/
1	为什么要用 GPU	Guide 1.1、1.2.1–1.2.2	cuda/m1_why_gpu/
2	第一个 CUDA 程序	Guide 2.1 全章、2.6	cuda/m2_first_kernel/
3	SIMT 执行	Guide 2.3.1–2.3.2、1.2.2.2	cuda/m3_simt/
4	存储空间	Guide 2.3.3–2.3.5、2.3.7、1.2.3	cuda/m4_memory/
5	计时与异步初步	Guide 2.5 引言、1.2.3.3	cuda/m5_async/
6	Tile 视角	Guide 1.2.2.3、2.4、2.2	(阅读为主)
7	TileLang 与 Triton	TileLang 文档、Triton 教程	kernels/ + tests/
8	平台与编译	Guide 1.3、2.1.1、2.7	(复用 cuda/m0_env)
Bonus	matmul	—	cuda/bonus/、kernels/

0 环境准备

先把环境跑通——编译一个 minimal 的 CUDA 程序，确认工具链和卡都能用；再把这块卡的参数查一遍，后面的 Module 会反复用到。环境配置见 README.md。

¹<https://docs.nvidia.com/cuda/cuda-programming-guide/>，2025 年重排的新版。下文引用记作 Guide。

参考资料 assignment01/README.md; 查硬件参数用 Guide 第五部分附录 Compute Capabilities。

prob 0.1 (HANDS-ON)

cuda/m0_env/01_hello.cu

编译并运行第一个程序，它启动 4 个 block、每个 block 8 个线程。

```
cd assignment01/cuda
make run/m0_env/01_hello
```

看看输出的结果，建议至少运行 5 次，可以留意输出各行顺序的变化。源码可以先不细读，Module 8 会回头看 nvcc 对它做了什么。

0.1 实验记录

运行 5 次，每次 32 行（4 个 block，每个 8 个线程）。第 1、2、4、5 次 block 输出顺序为 $1 \rightarrow 2 \rightarrow 0 \rightarrow 3$ ，第 3 次为 $2 \rightarrow 1 \rightarrow 0 \rightarrow 3$ 。观察到 block 输出顺序不固定。

prob 0.2 (FILL-IN)

cuda/m0_env/02_device_query.cu

02_device_query.cu 的五个空都是 cudaDeviceProp 的字段名，对照 CUDA Runtime API 文档补全，然后编译运行。

```
cd assignment01/cuda
make run/m0_env/02_device_query
```

把你这块卡的参数填进下表，并对照 Guide 附录 Compute Capabilities 里对应架构的表，核对 warp 大小和每 SM 最大线程数。

项目	你的卡
型号 / compute capability	NVIDIA GeForce RTX 3080 Ti / 8.6
SM 数量	80
warp 大小	32
shared memory / block	49152 字节 (48 KiB)
最大常驻线程 / SM	1536
显存总量	12489195520 字节 (约 11.63 GiB)

Editor's Note 本模块两道题看上去只是热身，但 prob 0.2 的那张表后面会反复被引用：Module 3 讨论 warp 要用 warp 大小，Module 4 手算驻留 block 数要用“shared memory / SM”与“最大常驻线程 / SM”。Keep it at hand.

另外，我们将会逐渐领悟到：一段在 GPU 上运行的代码，其性能与硬件型号是强绑定的。同一段代码在不同 compute capability 上的性能可以相差数倍，某些优化只在某几代 compute capability 上奏效。此后每记录一个耗时，请将硬件型号与 compute capability 一同记下。

1 为什么要用 GPU

GPU 与 CPU 的根本区别在设计目标——CPU 为单线程延迟服务，GPU 为总吞吐服务。此 Module 关注延迟/吞吐、SIMD/SIMT 等相关概念。

参考资料 Guide 1.1、1.2.1–1.2.2; LCPU 讲义^a Part 0。

^a<https://github.com/interestingLSY/CUDA-From-Correctness-To-Performance-Code/blob/master/lecture.md>

prob 1.1 (CONCEPT)

判断对错，可以顺带补一句理由。

- (a) 一块标称 100 TFLOPS 的 GPU，执行单条指令的延迟一定低于 5 GHz 的 CPU。
- (b) HBM 的“高带宽”指大块连续访问时的吞吐，零散的随机访问达不到标称值。
- (c) 严格串行的迭代算法（每步依赖上一步的结果），即使换一块算力更强的 GPU 也快不了多少。
- (d) “算力 1000 TFLOPS”意味着每次运算的延迟是 10^{-15} 秒。

1.1 作答（讨论后整理）

- (a) 错。GPU 能通过大量线程并发执行获得高吞吐；TFLOPS 是吞吐指标，GHz 是时钟频率，不能仅凭这两个数字判断单条指令延迟。
- (b) 对。零散访问可能触及更多缓存行或内存事务，而每次搬运的数据只有一部分被使用，有效数据利用率低，难以达到标称带宽。
- (c) 对。每一步依赖上一步的结果，无法充分并行，增加并行计算能力难以显著加速这条串行依赖链。
- (d) 错。TFLOPS 表示每秒浮点运算数，不是单次运算延迟。已知指令延迟周期数时，延迟等于周期数除以频率；例如 20 个周期、2 GHz 对应 10 ns。

prob 1.2 (CONCEPT)

Session 1 讲座里提过“N 方过百万”这个例子。总计算量 10^{12} FLOP 在当代 GPU 上的运算时间大概是毫秒级，那为什么一个严格在线的串行算法仍然做不到几秒内跑完？（从“延迟”和“吞吐”的角度考虑）

1.2 作答（讨论后整理）

大量独立运算在有足够并行度时，计算耗时可近似为总工作量除以实际达到的吞吐量。严格串行算法的下一步必须等待前一步结果，耗时取决于依赖链上各步延迟的累加；若每步延迟相同，可近似为步数乘以单步延迟。因此不能直接用 GPU 的峰值吞吐量估算严格串行算法耗时。

prob 1.3 (CONCEPT)

补全下表（thread 一行已填好作为示例）

执行层次	软件含义	对应硬件	直接可用的存储	同步与通信手段
thread	kernel 的最小执行单位	计算单元上的一个 lane	自己的寄存器	(自身天然有序)
warp				
block	/			
CTA				
grid				

1.3 已回答部分（其余待补）

一个 warp 包含 32 个线程，线程来自同一个 block，不能跨 block 组成 warp。线程不能像读取自己的变量一样直接读取另一线程的寄存器。已提出的通信方式：通过 shared memory 交换数据；使用时需要正确同步。一个 block 可以包含多个 warp，256 个线程对应 8 个 warp。一个 block 只能在一个 SM 上执行；在资源允许时，一个 SM 可以同时容纳多个 block。同一 block 中的线程可通过 shared memory 交换数据，并使用 `__syncthreads()` 进行 block 范围的同步。

一次 kernel 启动产生的所有 block 合起来构成一个 grid，各 block 可被调度到不同 SM。普通 grid 编程模型下，跨 block 共享的结果应放在 global memory；`__syncthreads()` 只同步本 block，不能让 block 0 等待 block 1。普通 block 之间不能依赖固定执行次序，这是 CUDA 可伸缩编程模型的约束。补充：SM90 及以后支持 thread block cluster 和 distributed shared memory，但这是显式使用的扩展层次，不改变普通 grid/block 题目的答案。

warp 是 SM 上的调度和指令发射单位，由 warp scheduler 从 ready warp 中选择并发射指令；它不是 Tensor Core。warp 内线程可用 `__shfl_sync` 一类 shuffle intrinsic 直接交换寄存器值。普通 grid 的全局阶段同步最简单的方式是结束当前 kernel，再在同一 stream 中启动下一个 kernel：前一个 kernel 的完成构成 kernel 边界。若必须在单个 kernel 内进行 grid-wide 同步，则需要 cooperative launch 等额外机制及其驻留约束。

prob 1.4 (CONCEPT)

SIMD 与 SIMT 的区别？另：判断正误——Nvidia GPU 在 Volta 之后每个线程有独立的 program counter，所以 branch divergence 不再有性能代价。

1.4 作答（讨论后整理）

SIMD 是一条指令显式作用于多个数据元素；SIMT 则让程序员编写和组织大量线程，硬件再把线程组成 warp，以带 active mask 的方式执行共同指令。题中判断错误。Volta 之后的 independent thread scheduling 为每个线程维护独立 program counter 和调用栈，改善了线程调度和同步能力，但没有消除 warp 的执行资源共享。当同一 warp 中连续的 `threadIdx.x` 因奇偶条件分别进入两条路径时会产生 branch divergence；硬件需要在不同 active mask 下执行各路径，因此仍可能产生额外代价。

prob 1.5 (EXPERIMENT)

`cuda/m1_why_gpu/01_scaling.cu`

同一个向量加法，四种配置各跑一遍。

```
cd assignment01/cuda
make run/m1_why_gpu/01_scaling
```

将数据填入下表，并回答：(a) GPU 单线程为什么比 CPU 慢这么多？(b) 从单 block 到铺满 grid 的提速，说明 GPU 加速计算靠的是什么？

配置	耗时 (ms)	ns / 元素
CPU 单线程	10.915	2.60
GPU <<<1,1>>>	256.533	61.16
GPU <<<1,256>>>	5.082	1.21
GPU 铺满 grid	0.071	0.02

1.5 补充记录

铺满 grid 的配置为 16384 blocks $\times$ 256 threads。表中 ns / 元素沿用程序打印精度，是总耗时除以元素数。GPU 计时不包含分配与 CPU/GPU 拷贝。

GPU 的设计目标是高吞吐而非最低单线程延迟。只启动一个 GPU 线程时，一个 warp 中其余 lane 没有有效工作，也没有足够的其他 warp 隐藏访存等延迟，因此比面向低延迟执行的 CPU 单线程慢。<<<1,256>>> 虽有 256 个线程，但一个 block 只能在一个 SM 上执行，无法使用 RTX 3080 Ti 的其余 79 个 SM。从单 block 的 5.082 ms 到铺满 grid 的 0.071 ms，说明 GPU 加速主要依靠大量线程和多个计算单元并行工作来获得高吞吐。启动全部线程不表示全部同时常驻。

prob 1.6 (FROM-SCRATCH · Optional)

kernels/simt_sim.py

SIMT Simulator ——写一个 warp 的执行模拟器：32 个 lane、共享控制流、带 mask 的分支执行与汇合。请在 kernels/simt_sim.py 内完成 (docstring 里面有具体实现要求)。工作量大概是 50 行以内的 Python。不需要 GPU。注：本题为 Module 3 的实验结果提供理论对照。判测命令如下：

```
cd assignment01
uv run pytest tests/test_simt_sim.py
```

Editor's Note 延迟与吞吐的分野根植于设计取舍，而非工艺差异。把“同一笔晶体管预算”投向不同的方向：一边是乱序执行、分支预测与庞大的 cache，另一边则是更多计算单元和更宽的访存带宽。prob 1.5 用四档配置让这个判断变得可度量，prob 1.1 与 prob 1.4 分别从两侧划出了它的边界，选做的 prob 1.6 则预先准备了一份理论标尺，留待 Module 3 的实测去校准。

更值得记住的是 prob 1.2 得出的那条结论：无论可用吞吐有多高，它始终绕不过一条串行依赖链。判断某个任务是否值得搬上 GPU，最先看的不是总计算量，而是依赖“图”的宽度——Amdahl 定律在 GPU 场景下的变体大抵如此。至于 prob 1.5 的那份向量加法，它几乎榨不出这块卡真正的算力，真正的瓶颈藏在另一个地方；Module 4 会为你把它量化出来。

2 第一个 CUDA 程序

此 Module 把一个 CUDA 程序的完整生命周期过一遍，覆盖 Guide 2.1 的每个小节。

参考资料 Guide 2.1 全章（主要出处）、2.6（浏览 unified memory 相关小节）；LCPU 讲义 Part 1。

prob 2.1 (FILL-IN)

cuda/m2_first_kernel/01_vector_add.cu

请填空：m2_first_kernel/01_vector_add.cu，具体要求见代码注释。判测命令如下：

```
cd assignment01/cuda
make run/m2_first_kernel/01_vector_add
```

2.1 判测记录

已完成填空；远程运行结果：PASS。下标理解检查：每 block 256 个线程时，block 2 的 thread 5 对应全局元素 $512 + 5 = 517$ 。

prob 2.2 (CONCEPT)

为下列五个场景选择正确的修饰符（如 `__global__` 等）。

- (a) 在 GPU 上执行、由 CPU 侧启动的 kernel 函数。
- (b) 只会被 kernel 调用的辅助函数。
- (c) host 和 device 代码都要调用的工具函数。
- (d) 整个 kernel 运行期间不变、所有线程都要读的系数表。
- (e) block 内线程共享的暂存数组。

2.2 作答（讨论后订正）

- (a) `__global__`：在 device 上执行，由 host 发起 kernel launch。
- (b) `__device__`：在 device 上执行，并由 device 代码调用。
- (c) 同时使用 `__host__` 和 `__device__`，分别生成 host 与 device 版本；`inline` 不是决定执行位置的修饰符。
- (d) `__constant__`，用于 device constant memory 中的只读系数表。
- (e) `__shared__`，用于一个 block 内线程共享的暂存数据。

prob 2.3 (MODIFY)

cuda/m2_first_kernel/02_vector_add_um.cu

02_vector_add_um.cu 代码完整，但目前是“显式内存管理”的版本。改之前先按原样编译运行一次，记下耗时——这一版会被你的改动覆盖掉，下面 (b) 要拿它做对照。然后按文件头的说明改成 unified memory 版 (cudaMallocManaged)，并保持文件头写明的计时窗口不变。

```
cd assignment01/cuda
make run/m2_first_kernel/02_vector_add_um
```

然后请回答如下问题：(a) kernel 启动之后、CPU 读结果之前，为什么必须有一次同步？在原先的版本里这次同步发生在哪个调用里？(b) 对比两版“搬运 + kernel + 读回”的耗时，分析差距的原因（谁快谁慢都有可能，与使用的卡有关）。

2.3 实验记录

已将两组数组改为同一组 managed 指针，CPU 初始化、kernel 和 CPU 校验均使用该组指针，去掉显式拷贝。调整错误检查宏的用法后，UM 版本判测为 PASS。分配和初始化保持在计时外，CPU 完整遍历结果保持在计时内。

旧容器显式管理基准：94.3 ms（不用于跨容器直接比较）。

新容器 UM 全部测量 (ms, 按运行顺序)：526.0、221.7、173.8、225.9、119.5、71.9、73.0、76.6、75.4、73.7。各次均为 PASS。

新容器后续对照组 (ms)：94.1、100.9、96.2、97.7、93.0、95.9、106.5、95.8。各次均为 PASS。版本待确认：该组按对话中的“切回原版”步骤暂记为显式管理版，终端输出本身未标注版本。UM 最后 5 次范围 71.9–76.6 ms，中位数 73.7 ms；对照组全部 8 次中位数 96.05 ms。在上述版本归属成立的前提下，这两组中位数对应约 23.3% 的耗时降低（约 1.30 倍加速）。UM 前期波动明显，此处仅为后期样本比较，不能代表全部运行或证明具体性能原因。

同步理解检查已回答：不等待 kernel 完成，CPU 可能读到错误结果。原程序中的 `CUDA_CHECK_KERNEL()` 宏内部已经调用 `cudaDeviceSynchronize()`，这是代码中实际最先发生的同步；即使删掉该宏中的同步，后续同步的 device-to-host `cudaMemcpy` 也会等待同一 stream 中排在它之前的 kernel。

UM 可能更慢，因为 CPU 初始化后，GPU 首次访问 managed memory 页面可能触发 page fault，并将输入页从主机迁移到 GPU；GPU 写完结果后，CPU 遍历输出又可能触发页面向主机迁移。按需缺页处理和较细粒度迁移会带来额外管理开销。UM 也可能在特定平台和访问模式下更快，例如运行时的迁移策略可能减少普通 pageable host memory 显式拷贝所需的 staging 开销；测量噪声和系统状态也会影响本实验。当前总耗时不足以确定本机差异的具体来源，因此结论仅限于所测样本，不推广为 UM 总是更快。

prob 2.4 (CONCEPT)

判断对错，可以顺带补一句理由。

- (a) `vectorAdd<<<...>>>(...)` 这条语句返回时，kernel 一定已经执行完毕。
- (b) 同一个 stream 里，`cudaMemcpy` (device 到 host) 会等它前面的 kernel 全部完成后才开始拷贝。
- (c) kernel 内部的非法访存，会在启动语句处同步地报出来。

2.4 作答（讨论后整理）

- (a) 错。普通 kernel launch 对 host 是异步的，启动语句返回不表示 kernel 已完成。
- (b) 对。同一 stream 中按提交顺序执行，同步的 device-to-host cudaMemcpy 会等待前面的 kernel，并在拷贝完成后返回。
- (c) 错。kernel 内非法访存属于执行期错误，启动语句通常已先返回；错误一般在后续同步调用处暴露。启动配置等 launch error 可用 cudaGetLastError() 检查，而执行期错误还需要同步才能可靠发现。本仓库的 CUDA_CHECK_KERNEL() 同时完成这两类检查。

prob 2.5 (DEBUG)

cuda/m2_first_kernel/03_bug_launch.cu

修 bug: 03_bug_launch.cu, 详细内容见相关文件。

```
cd assignment01/cuda
make run/m2_first_kernel/03_bug_launch
```

2.5 调试记录

已提供的原始报错：

```
CUDA error cudaErrorInvalidValue at
m2_first_kernel/03_bug_launch.cu:35: invalid argument
```

原配置为每 block 2048 个线程，超过设备的每 block 最大线程数 1024，因而启动失败。这里的 1024 不应与每 SM 最大常驻线程数 1536 混淆：前者限制单个 block 的启动形状，后者限制一个 SM 同时驻留的所有 block/thread。CUDA Runtime 将错误状态返回或记录下来，并不会在没有检查的情况下自动替程序打印错误；需要调用 cudaGetLastError()、同步 API 或错误检查宏来观察它。启动前 d_c 已被清零，kernel 未执行就不会写入加法结果；复制回 host 的零数组与非零参考结果不一致，因此对拍输出 FAIL。调整 block 大小后重新运行，结果为 PASS。

prob 2.6 (FILL-IN)

cuda/m2_first_kernel/04_matrix_add.cu

04_matrix_add.cu 用二维的 block 和 grid 处理 1000×700 的矩阵，请填空实现矩阵加法。测试命令：

```
cd assignment01/cuda
make run/m2_first_kernel/04_matrix_add
```

prob 2.7 (MODIFY)

cuda/m2_first_kernel/05_grid_stride.cu

05_grid_stride.cu 的 launch 被固定成 <<<64, 256>>>，线程总数远小于 n ，当前 FAIL。在 launch 配置不变的前提下，把 kernel 改成 grid-stride loop，让任意 n 都能 PASS。

```
cd assignment01/cuda
make run/m2_first_kernel/05_grid_stride
```

然后请回答——这种写法的价值在哪里？launch 只有 16384 个线程时，性能上要付出什么代价？

2.7 判测记录

已改为 grid-stride loop, 保留 `<<<64,256>>>` 启动配置; 远程判测为 PASS。本地代码中每轮步长为 `blockDim.x * gridDim.x`。这种写法把 launch 的 block 数与数据规模解耦: 数据增加时不必按元素数同比增加 block, 每个线程可循环处理后续元素, 因此同一 kernel 能覆盖不同大小的输入并复用线程。代价是并行度可能不足。本机有 80 个 SM, 而整个 grid 只有 64 个 block, 至少有 16 个 SM 无法获得该 kernel 的 block; 每个有工作的 SM 也只有一个 256-thread block (8 个 warp), 可能不足以充分隐藏延迟。每个线程还需循环处理约 1024 个元素, 因此相较铺满 grid 的方案会增加串行工作量和循环开销。

prob 2.8 (EXPERIMENT)

cuda/m2_first_kernel/06_whoami.cu

运行下面的程序两三次, 观察 16 个 block 打印输出的先后。

```
cd assignment01/cuda
make run/m2_first_kernel/06_whoami
```

(a) 顺序由谁决定? (b) 程序的正确性可以依赖 block 的执行顺序吗? 这条限制和 Guide 1.1 说的 scalable programming model 有什么关系?

2.8 作答 (讨论后整理)

各 block 的分配和执行时机由 GPU 调度器决定, 程序中的 thread 0 只负责打印观察结果, 并不决定 block 顺序; device printf 的缓冲机制也会影响屏幕上看到的顺序。程序正确性不能依赖普通 grid 中 block 的先后顺序。正因为 block 相互独立且调度顺序不受程序假定, 同一个 grid 才能在 SM 数量不同的 GPU 上执行: 硬件可按可用资源分批调度 block, 这体现了 scalable programming model。

prob 2.9 (FROM-SCRATCH): SAXPY

cuda/m2_first_kernel/saxpy.cu

在 m2_first_kernel/ 下写出完整的 CUDA 程序 saxpy.cu, 实现 $y \leftarrow 2.0 \cdot x + y$ (单精度)。要求如下:

- 不允许 include common.h。错误检查宏和 cudaEvent 计时都要自己写一遍。
- 命令行用法: `./saxpy <n>`, n 是元素个数。输入数据按固定公式生成 (都是 float): $x[i] = ((i \% 2048) - 1024) * 0.5f$, $y[i] = (i \% 1024) - 512$ 。
- kernel 算完把 y 拷回 host, 用 double 累加所有 $y[i]$, 输出一行 SUM=< 总和 > (用 `printf("SUM=%.0f\n", s)` 这样的格式, 同一行里可以再带上 n 和 kernel 毫秒数), SUM 结果将用于对拍检验程序正确性, exit code 应为 0。
- $n = 0$ 时输出 SUM=0, exit code 为 0 (0 个 block 的 kernel launch 是非法的, 特判即可)。

判测脚本覆盖 $n \in \{0, 1, 31, 1024, 1025, 2^{20}, 2^{20}+3\}$, 命令如下。

```
cd assignment01/cuda/m2_first_kernel
./judge_saxpy.sh saxpy.cu
```

注: SAXPY 即 Single-precision A·X Plus Y。

Editor's Note CUDA C++ 在 C++ 之上添加的语法扩展其实相当克制：几种函数与变量修饰符、一个 kernel 启动语法，以及一组内存管理 API。真正需要你花时间去适应的是另外两件事——host 与 device 各自独立推进的时间线 (prob 2.4)，以及数据此刻落在谁的地址空间里，你得“显式”地负责 (prob 2.3)。作为本模块收尾的 prob 2.9 把这些概念连同索引、边界检查与错误处理一起收进一个不到百行的程序，prob 5.1 还会回过头来检验其中的计时部分。

prob 2.7 和 prob 2.8 值得多花一些时间。你为这两道题写下的理由——一条关于启动配置与问题规模的关系，另一条关于 block 之间的执行顺序——在后面的内容里会以不同面貌反复出现。Module 7 介绍的 Triton 与 TileLang 仍然建立在同一条 block 无序执行的假设之上，只不过不再需要你亲手把它写出来。

3 SIMT 执行

SIMT 给每个线程“独立执行”的表象，硬件却按 32 线程一组共享 instruction fetch 和 issue (取指和发射)。此模块三个实验的主题分别为“divergence 的代价”、“__syncthreads 的必要性”、“block 之间为什么无法同步”。

参考资料 Guide 2.3.1–2.3.2、1.2.2.2。3.3、3.5 要用 shared memory，用法见 Guide 2.3.3.2 与 1.2.3.2 (Module 4 会系统地读这一块)；cooperative groups 见 Guide 2.3.6 (只需浏览)。

prob 3.1 (CONCEPT)

设 `blockDim = (8, 8, 1)`。

- `threadIdx = (3, 5, 0)` 的线性编号是多少？它在第几个 warp、warp 内第几个 lane？
- 这个 block 一共占多少个 warp？
- 若 `blockDim = (33, 1, 1)`，占几个 warp？这样配置浪费在哪里？

3.1 作答 (讨论后整理)

二维 block 的线性线程编号为 $3 + 5 \times 8 = 43$ 。按 0 起始编号，它位于 warp $\lfloor 43/32 \rfloor = 1$ ，lane $43 \bmod 32 = 11$ ，即口语所说的第 2 个 warp。 $8 \times 8 = 64$ 个线程恰好占 2 个 warp。33 个线程也要占 2 个 warp，其中第二个 warp 只有 1 个 active lane，剩余 31 个 lane 没有有效工作。

prob 3.2 (EXPERIMENT)

`cuda/m3_simt/01_divergence.cu`

`m3_simt/01_divergence.cu` 的两个 kernel 每线程计算量相同，分支划分不同——一个按 thread 编号的奇偶分 (同一个 warp 里一半一半)，一个按 warp 边界对齐分。请先预测一下哪个版本运行会更快一点，大概快几倍，然后运行验证：

```
cd assignment01/cuda
make run/m3_simt/01_divergence
```

请解释实测比值，并回答——若两个分支的计算量一大一小，按 thread 编号奇偶分的 kernel 和按 warp 边界对齐分的 kernel 的运行时间分别由什么决定？

3.2 实验记录（部分完成）

预测按 warp 边界对齐的分支更快，约为奇偶线程分支的 2 倍，因为奇偶分支时每条路径只有约 50% 的 lane active。实测：warp 内奇偶分支 0.779 ms，按 warp 分支 0.392 ms，比值 1.99，与预测吻合。奇偶分支使同一 warp 在不同 active mask 下分别执行两条路径；按 warp 对齐时，每个 warp 只走一条路径并保持全部 lane 有效。若路径 A、B 的工作量分别为 100 和 20 次循环，奇偶分支的每个 warp 需要在不同 mask 下执行两条路径，工作成本近似由二者之和 120 决定；按 warp 对齐时，A warp 和 B warp 分别执行 100 和 20 次，A warp 更晚完成。整个 kernel 的实际耗时还受两类 warp 的数量、调度和资源利用率影响。

prob 3.3 (EXPERIMENT)

cuda/m3_simt/02_sync_matters.cu

02_sync_matters.cu 让每个 block 用 shared memory 把自己的 256 个元素倒序。请按文件开头的注释内容进行实验。

```
cd assignment01/cuda
make run/m3_simt/02_sync_matters
```

(a) 为什么注释掉 sync 后代码不能正确地运行？(b) (Optional) 注释掉 sync 后，翻转后的数组错的位置比较随机，但是有些位置一直是对的，试解释原因。（tip: 算一算 t 与 $255 - t$ 有没有可能落在同一个 warp）

3.3 实验记录与作答

保留 `__syncthreads()` 时重复运行得到 PASS。去掉同步后运行出现 MISMATCH at 64: got 0, want 8.53, 另一次出现 MISMATCH at 128: got 0, want 4.89, 最终均为 FAIL。原因是各线程写完 `buf[t]` 后需要读取另一线程负责写入的 `buf[255-t]`；没有 block barrier 时，读取方所在 warp 可能先于写入方所在 warp 执行读取，从而读到尚未写好的 shared memory 内容。实验中一次 `undefined reference to main` 是修改注释范围时产生的独立编译错误，不计入同步实验结论。

选做分析： $t = 0$ 与 $255 - t = 255$ 分别位于 warp 0 和 warp 7； $t = 100$ 与 155 分别位于 warp 3 和 warp 4。一般地，若 t 位于 warp w ，则 $255 - t$ 位于 warp $7 - w$ ，不存在整数 $w = 7 - w$ ，因此两者不可能位于同一 warp。若某个 warp 较晚执行读取，它要读取的位置可能已经被较早执行的对应 warp 写好，因此部分输出会碰巧正确；这种结果依赖实际调度，不能作为正确性保证。

prob 3.4 (CONCEPT)

`__syncthreads` 只能同步本 block 内的 threads，那需要全 grid 同步时，标准做法是什么？

3.4 作答（复用前述讨论）

标准做法是结束当前 kernel，并在同一 stream 中启动下一阶段 kernel；kernel 边界保证后一 kernel 在前一 kernel 完成后执行。若必须在单个 kernel 内进行 grid-wide 同步，则需要 cooperative launch 和 cooperative groups 等专门机制，并满足相应驻留约束。

prob 3.5 (FROM-SCRATCH): block 内归约

cuda/m3_simt/03_reduce.cu

在 03_reduce.cu 里从零实现两个求和归约 kernel (判测与计时的代码已经写好)。两个 kernel 的 contract 见文件头。PASS 后, 试解释实测性能差距的原因。

(Optional) 基于两点事实——(a) `__shfl_down_sync` 是 warp 内寄存器级别的线程间数据交换指令, 自带同步效果且延迟极小; (b) 归约到最后 32 个元素后, 活跃线程若都落在同一个 warp 里, 就不再需要 `__syncthreads` (可以想想为什么)——据此试写出第三版优化后的 kernel。测试时只会跑前两版, 第三版自己在 main 里照着加一次 `run_one` 调用即可。

```
cd assignment01/cuda
make run/m3_simt/03_reduce
```

3.5 实验记录与分析

三个实现均通过正确性检查。修正统计方向后的 RTX 3080 Ti 最终实测: interleaved 平均 0.0239 ms, contiguous 平均 0.0171 ms, shuffle 平均 0.0105 ms。程序报告 interleaved/contiguous 为 1.40 倍, interleaved/shuffle 为 2.27 倍; 由数据另算, contiguous/shuffle 约为 $0.0171/0.0105 = 1.63$ 倍。interleaved/contiguous 比值低于程序的 1.5 倍提示阈值, 但仍显示 contiguous 更快; 提示阈值所列 A100、V100 参考值不能视为不同架构 GPU 上必须达到的正确性条件。

两版的加法次数和循环轮数相同, 差异在 active lane 的分布。interleaved 第一轮中, 8 个 warp 都要发射带掩码的加法指令, 但每个 warp 只有偶数 lane 工作; contiguous 第一轮中, 前 4 个 warp 的所有 lane 工作, 后 4 个 warp 不执行加法。后续各轮中, contiguous 继续把 active lane 集中在更少的 warp 内, 因此提高执行加法指令时的 lane 利用率, 并减少需要执行该指令的 warp 数。interleaved 的运行期取模条件也可能带来额外指令成本。这里减少的不是循环轮数, 而是低利用率的 warp 指令工作。

shuffle 版先在 shared memory 中把 256 个输入归约为集中在 warp 0 的 32 个部分和, 再用 `__shfl_down_sync` 在 warp 内直接交换寄存器值完成最后阶段。它减少了后续 shared memory 访问和 block-wide barrier, 因此比完整的 shared-memory contiguous 版更快。

Editor's Note SIMT 让你以单线程的风格编写并行代码, 但它只承诺正确性语义, 不负责性能语义。本模块的题目正好分别对应这层抽象漏出来的三个口子: warp 才是真实的调度单位 (prob 3.2), block 内的并行需要显式同步才能看到彼此的写入 (prob 3.3), 而 block 之间压根没有提供同步方法 (prob 3.4)。

“For performance reasoning, you always need to fall back to the warp level.”

prob 3.5 抛出的结论更让人不适: 两个版本的加法次数完全相同, 耗时却能相差超过一倍。算法复杂度在这里不再是性能的充分描述——这大概是 GPU 编程与经典算法课之间最根本的分歧。选做的第三版则指向另一条路径: 不再把 warp 当作需要小心绕开的硬件细节, 而是直接把它当作可调用的编程接口。warp-level primitive 与 cooperative groups (Guide 2.3.6) 正是这个方向的官方解答。

4 存储空间

数据的存储位置和迁移速度极大地影响着 kernel 的速度。此 Module 旨在对“存储如何影响性能”建立起基础认知。

参考资料 Guide 2.3.3–2.3.5、2.3.7、1.2.3。

prob 4.1 (CONCEPT)

补全下表：

空间	谁可见	生命周期	片上 / 片外	谁管理
register	单个线程	线程	片上	编译器
local	单个线程	线程	片外	编译器
shared	单个 block	block	片上	程序员声明/组织
global	所有 device 线程 (host 经 API 访问)	从分配到释放	片外	程序员与 CUDA Runtime
constant	所有 device 线程 只读	模块/应用期间	片外, 片上缓存	程序员经 API 写入
L1 / L2 cache	L1 每 SM; L2 全 GPU 共享	硬件按需	片上	硬件

prob 4.2 (FILL-IN)

cuda/m4_memory/01_stencil.cu

请填写：m4_memory/01_stencil.cu，具体要求见代码注释。测试命令：

```
cd assignment01/cuda
make run/m4_memory/01_stencil
```

4.2 判测记录

static shared memory 版本与 dynamic shared memory 版本均通过判测，最终输出 PASS。本次运行来自新容器，GPU 型号待重新确认。

prob 4.3 (MODIFY)

cuda/m4_memory/02_constant_coeff.cu

02_constant_coeff.cu：按文件头的说明进行修改。修改完后测试：

```
cd assignment01/cuda
make run/m4_memory/02_constant_coeff
```

结果会包含两个版本的耗时（差距可能极小，甚至测不出来性能收益——想想为什么），回答 constant cache 真正的优势在哪种访问模式。

4.3 实验记录与性能分析

RTX 3080 Ti 实测: global 版本 0.1642 ms, constant 版本 0.1649 ms, 程序报告 global/constant 为 1.00 倍; 两版均为 PASS。constant 版在该次测量中约慢 0.4%, 差异很小, 不足以视为有意义的性能变化。

Horner 循环执行到同一个 k 时, 一个 warp 的 32 个线程都读取同一个 `coef[k]`, constant cache 可以把该值一次广播给整个 warp。constant cache 的主要优势正是 warp 内线程读取相同或少数几个地址的访问模式。这里系数表只有 8 个 float, 工作集总计 32 B, global-memory 版本在首次访问后也很容易从 cache 命中; 同时 kernel 还需读写大数组并执行算术, 因此把这张极小的表换到 constant memory 没有改变主要瓶颈, 实测收益不明显。

prob 4.4 (CONCEPT)

判断对错, 可以顺带补一句理由。

- (a) local memory 的“local”指作用域私有, 它实际上在片外显存里。
- (b) 对数组用运行期才知道的下标做索引, 可能迫使它被放进 local memory。

4.4 作答 (讨论后整理)

两项均正确。这里的“片”指 GPU silicon die, 并非整个 GPU 设备或显卡: register、shared memory 和 cache 等 SRAM 位于计算芯片上; global/local memory 的主体位于芯片外的 GDDR/HBM 显存芯片中, 但显存仍属于 GPU device。local memory 的“local”描述单线程私有的地址空间, 不描述物理距离。线程私有数组若采用运行期索引, 编译器可能无法将各元素静态映射到寄存器, 因而放入 local memory; 寄存器数量不足造成的 register spilling 也会落到 local memory。local memory 物理上通常由 device memory 支撑, 并可经过 cache。

prob 4.5 (FILL-IN)

cuda/m4_memory/03_histogram.cu

请填空: 03_histogram.cu, 具体要求见代码注释。测试命令:

```
cd assignment01/cuda
make run/m4_memory/03_histogram
```

4.5 判测与基准记录

RTX 3080 Ti 上运行 naive global-atomic 直方图, 平均耗时 4.7255 ms, 有效输入吞吐 3.55 GB/s, 正确性判测为 PASS。该数据作为 4.6 私有化版本的独立参考; 4.6 程序还会在同一次运行中重新测量两版, 以该同进程结果作为正式性能对比。

prob 4.6 (MODIFY)

cuda/m4_memory/04_histogram_priv.cu

04_histogram_priv.cu: 请按要求修改代码。改完测试指令:

```
cd assignment01/cuda
make run/m4_memory/04_histogram_priv
```

测试结果包含与 naive 实现相比较的耗时、吞吐。试解释提速来自哪里。

4.6 实验记录与分析

RTX 3080 Ti 同进程实测：naive 版本 4.8285 ms、3.47 GB/s；shared-memory 私有化版本 0.0611 ms、274.46 GB/s；两版均为 PASS，naive/priv 耗时比为 78.99 倍。

naive 版共执行 $2^{24} = 16,777,216$ 次 global atomic；若输入均匀，每个 bucket 约承受 $2^{16} = 65,536$ 次。私有化版主统计阶段仍有同样数量的 atomic，但它们发生在每个 block 独立的片上 shared histogram 中，消除了不同 block 在此阶段对同一组 global bucket 的竞争。最终合并只执行 $1024 \times 256 = 262,144$ 次 global atomic，即每个 bucket 1024 次；与 naive 的每桶 65,536 次相比，global atomic 数和每桶 global 更新次数均减少约 64 倍。

shared atomic 延迟较低，竞争被限制在单个 block 内，global atomic 的长队列也大幅缩短，因此获得显著提速。78.99 倍不必等于 64 倍，因为 atomic 次数并不是完整的线性耗时模型：两版还包含输入读取、shared histogram 清零、同步和最终合并等不同成本，原子操作的争用与硬件调度也可能呈非线性，测量还存在波动。打印的 GB/s 只按输入字节数计算，是有效吞吐指标，并非全部物理内存流量。

prob 4.7 (EXPERIMENT)

cuda/m4_memory/05_bandwidth.cu

运行实验，并根据实验数据填表。

```
cd assignment01/cuda
make run/m4_memory/05_bandwidth
```

观察数据变化趋势，并简析趋势的成因。

stride	1	2	4	8	16	32
GB/s	815.4	549.8	338.7	188.4	188.4	187.9

4.7 实验记录与趋势分析

RTX 3080 Ti 实测耗时 (ms)：stride 1、2、4、8、16、32 分别为 0.1646、0.2441、0.3962、0.7123、0.7123、0.7142；程序按每个元素 4 B 读加 4 B 写计算的有效带宽分别为 815.4、549.8、338.7、188.4、188.4、187.9 GB/s。

stride 1 时，同一 warp 读取连续的 float，可合并成少量内存事务，所搬数据基本都被使用。stride 8 时，相邻线程的读取地址相隔 $8 \times 4 = 32$ B，近似使每个线程各自触及一个 32 B sector，却只使用其中 4 B，读事务的有效利用率为 12.5%。stride 继续增大时，一个线程已无法触发比单独一个最小事务更碎片化的读取，因此带宽在 stride 8 后接近平台。

输出 `out[i]` 的写入始终连续且可合并，所以程序整体指标不能只用读取侧的 12.5% 估算。粗略按每元素 32 B 物理读取加 4 B 合并写入，而程序分子计作 8 B 有效读写，利用率约为 $8/(32+4) = 22.2\%$ ； $815.4 \times 22.2\% \approx 181$ GB/s，与约 188 GB/s 的平台值接近。cache、事务合并细节以及取模映射造成的数据复用会带来偏差。

prob 4.8 (EXPERIMENT)

cuda/m4_memory/06_occupancy.cu

06_occupancy.cu，详细内容见相关文件。实验命令如下：

```
cd assignment01/cuda
make run/m4_memory/06_occupancy
```

记录实验数据：

shared memory / block (KB)	0.0	13.2	15.0	18.0	29.0	55.0
理论驻留 block / SM	6	6	6	5	3	1
occupancy	100.0%	100.0%	100.0%	83.3%	50.0%	16.7%
实测带宽 (GB/s)	833.6	833.7	833.8	836.5	812.9	410.2

4.8 实验记录与分析

RTX 3080 Ti: shared memory 100 KB/SM, 最大常驻 1536 threads/SM; 固定 block size 为 256。各档数据见上表。`cudaOccupancyMaxPotentialBlockSize` 在 dynamic shared memory 为 0 时建议 block size 768。本机实际 occupancy 档位为 100.0%、83.3%、50.0%、16.7%，因此题目 (c) 应依据这些档位分析，而非套用讲义中面向其他设备的 75%、37.5%、12.5%。以 29 KB/block 档为例，shared-memory 限制为 $\lfloor 100/29 \rfloor = 3$ blocks/SM，线程限制为 $1536/256 = 6$ blocks/SM；同时满足所有资源约束要取较小值，因此最终为 3 blocks/SM。occupancy 为 $3 \times 256/1536 = 50\%$ 。每个 256-thread block 含 8 个 warp，所以该档为 24 个常驻 warp/SM；满 occupancy 对应 $1536/32 = 48$ 个 warp/SM。

occupancy 从 100% 降到 83.3% 时，带宽由约 833.8 变为 836.5 GB/s，并未出现有意义的下降；block size 始终为 256，该约 0.3% 差异应视为测量波动，不能解释为 block 变小或调度开销下降。50% occupancy 时仍有 24 个常驻 warp/SM，当一个 warp 等待显存时，调度器还有足够的 ready warp 可切换，从而隐藏延迟并维持 812.9 GB/s。降到 16.7% 时只剩 8 个常驻 warp/SM，可用于重叠显存延迟的候选 warp 不足，带宽降至 410.2 GB/s，较 50% 档下降约 49.5%。因此 occupancy 是隐藏延迟、提供足够在途工作的手段；达到饱和所需的阈值后，继续提高 occupancy 未必带来性能提升，性能也不会与 occupancy 成比例变化。

请回答：(a) 用程序开头打印的“shared memory / SM”和“最大常驻线程 / SM”，手算其中一个的驻留 block 数和 occupancy，和 API 的结果对照。(b) 带宽为什么随 occupancy 下降？用“延迟隐藏需要足够多的常驻 warp”组织你的解释。(c) 表中带宽随 occupancy 单调下降，但明显不成正比——从 100% 到 75% 带宽掉了多少？从 37.5% 到 12.5% 又掉了多少？试解释这个差别。

Editor's Note 本模块的八道题围绕同一个问题展开：数据放在哪一层，以什么模式去取。prob 4.1 给出一幅静态的存储层次地图，prob 4.2 与 4.3 在同一段 stencil 上切换存储位置，prob 4.5 与 4.6 在同一个直方图统计上做同样的变换，prob 4.7 和 4.8 则分别量化访问模式与可并发的请求数量。

有三点值得单独记下。其一，优化只在真正的瓶颈处生效。prob 4.3 是一个刻意安排的负结果：教科书罗列的手段用在了不构成瓶颈的地方，收益为零。避免这类空转，需要先估算再动手，估算可以依赖 arithmetic intensity 与 roofline 模型，后面的 session 会展开。其二，你在 prob 4.8 (b) 写下的那句解释有一个正式的名称——Little 定律：维持某一吞吐量所需的在途请求数，等于吞吐与延迟的乘积。其三，occupancy 本身是手段，不是目标。它高只说明调度器有

足够多的 warp 可以切换，并不自动保证访存效率；反过来，低 occupancy 配合足够的指令级并行，同样能跑满内存带宽。Volkov 的 *Better Performance at Lower Occupancy* (GTC 2010) 是这一观点的经典论述。

5 计时与异步初步

在前面的练习里 `cudaDeviceSynchronize` 反复出现以保证计时的正确性。本 Module 主要关注两点——kernel 启动是异步的；以及如何准确计时。

参考资料 Guide 2.5（只读引言）、1.2.3.3。

prob 5.1 (EXPERIMENT)

`cuda/m5_async/01_timing_trap.cu`

运行实验：

```
cd assignment01/cuda
make run/m5_async/01_timing_trap
```

并回答下列问题：(a) 哪个数值可以当作 kernel 耗时写进报告？(b) 另外两个各具体测的是什么？

5.1 实验记录与作答

RTX 3080 Ti 实测：host 计时且不等待 GPU 为 0.0038 ms；host 计时并等待 GPU 为 0.1977 ms；CUDA event 计时为 0.1966 ms。报告 kernel 时间应采用 CUDA event 的 0.1966 ms。不同步的 host 计时主要测到 CPU 提交异步 kernel launch 的时间，并未等待计算完成。同步的 host wall-clock 计时覆盖 launch 提交、GPU 执行以及进入同步调用、host 等待和计时器等开销，所以与 kernel 时间接近但略有额外 host 成本。CUDA event 被记录在 GPU stream 时间线上，可排除大部分 host 提交和调度开销，因此更适合测量设备工作。

prob 5.2 (CONCEPT)

判断对错，可以顺带补一句理由。

- (a) 同一个 stream 里的操作按提交顺序执行。
- (b) kernel 启动后，host 代码立刻继续往下执行。
- (c) unified memory 下，CPU 访问一页正被 GPU 占用的内存，会触发缺页与页迁移。

5.2 作答（讨论后整理）

三项均正确。同一 stream 内的操作具有提交顺序，后续操作不会越过前面的操作完成其依赖工作；普通 kernel launch 对 host 是异步的，提交后 host 通常继续执行，除非遇到同步点；Unified Memory 需要维持 CPU/GPU 访问的一致性，CPU 访问当前由 GPU 占用或驻留在 GPU 侧的页面时，可能触发同步等待、缺页处理和页面迁移。

Editor's Note Module 5 的两道题分量不算重，但在整个认知链条里恰好落在一个关键的位置。CUDA 里的异步并不是某种需要额外开启的特性，它更像是 kernel launch 的默认语义：你把工作提交到某条 stream 上，host 端立刻返回，device 端的执行则在另一条时间线上独立展开。正因如此，“计时”这件事本身变成了一门需要认真对待的事——warmup、同步点插在哪里、选哪种 timer、重复多少次，任何一环处理得马虎，拿到的数字就失去了意义。这次作业里，这些细节已经由写好的测试框架替你包揽了；等到将来你自己搭建 benchmark，就得把这些一一补回来。

再下一步，异步自然会引出重叠：多 stream、数据拷贝与计算的并发、用 CUDA Graph 把整张任务图一次性提交。这些内容会放在后续 session 里展开，这里只需要建立起一个清晰的 mental model：host 只管提交，device 只管执行，两条时间线各自推进。

6 Tile 视角

Guide 从 13.x 起把 tile 编程作为与 SIMT 并列的第二种官方模型写进正文。tile 模型描述的是“一个 block 对一块数据做什么”，而 block 内 threads 的分工由编译器决定。此 Module 只做概念铺垫和对照阅读。

参考资料 Guide 1.2.2.3（必读，篇幅不长）、2.4.1–2.4.6（浏览）、2.2（知道 CUDA Python 这条路线存在即可）。

prob 6.1 (CONCEPT)

判断对错，可以顺带补一句理由。

- (a) tile 是显存里的一块可变区域，kernel 通过指针直接改写它。
- (b) 对 tile 的一次运算（如两个 tile 相加）由编译器映射到 block 内的多个线程上执行。
- (c) tile 模型与 SIMT 模型互斥，一个 CUDA 程序只能选一种。

6.1 作答（讨论后整理）

(a) 错。tile data 是不可变的值；一次 tile 运算会产生新的 tile。需要区分 tile data 与指向内存的 tiled view：前者不是由 kernel 原地修改的显存区域，向内存写回要通过 view 的 store 操作。(b) 对。程序描述整块数据上的运算，编译器负责把它映射为 block 内多个线程的底层执行。(c) 错。tile 与 SIMT 是可以共存、混合使用的编程模型，并不要求整个 CUDA 程序只能选择其中一种。

prob 6.2 (CONCEPT)

下面是 Guide 2.4.6 的 cuTile Python 向量加法：

```
import cuda.tile as ct

@ct.kernel
def vec_add(a, b, c, TILE: ct.Constant[int]):
    a_view = a.tiled_view((TILE,))
```

```

b_view = b.tiled_view((TILE,))
c_view = c.tiled_view((TILE,))

bid = ct.bid(0)
a_tile = a_view.load((bid,))
b_tile = b_view.load((bid,))
c_view.store((bid,), a_tile + b_tile)

```

据此补全下表（Triton 一行可以做完 Module 7 再回来填）：

	CUDA SIMT	cuTile	Triton
并行单位	block 里的 thread	block	
编号	blockIdx threadIdx	/	
数据分工	线程用全局下标来 划分数据		
边界处理	if 判断		

6.2 cuTile 部分作答（讨论后整理）

cuTile 的并行单位是 block，当前 block 在第 0 维的 tile-space 编号由 `ct.bid(0)` 取得。`tiled_view((TILE,))` 把一维数组划分为长度为 `TILE` 的连续区域，第 b 个 block 通过 `load((b,))` 处理下标 $b \cdot \text{TILE}$ 至 $(b + 1) \cdot \text{TILE} - 1$ ；例如 `TILE=256`、 $b = 3$ 时对应下标 768–1023。程序只描述 block 与 tile 的对应关系，tile 内元素到实际线程的映射由编译器完成。边界通过 tiled view 的边界语义处理，而不是在源码中为每个线程显式写 `if`：部分越界的 load 按 view 的 `padding_mode` 填充越界元素，store 忽略越界位置。若算法需要确定的越界读取值，应明确选择如 `PaddingMode.ZERO`；这与 Triton 源码中常见的显式 `mask` 参数形式不同。Triton 一行留待 Module 7 补充。

prob 6.3（CONCEPT）

仍看上面这段代码。(a) “每个线程对应哪个/些元素”由谁决定？(b) 列出一些在 CUDA SIMT 版向量加法里一定会出现、这里完全没体现出的概念。

6.2–6.3 作答（讨论后整理）

在 cuTile Python 中，`ct.bid(0)` 取得第 0 维的 block 编号；若 `TILE=256`，编号为 3 的 block 负责数组下标 3×256 到 $4 \times 256 - 1$ ，即 768–1023。`tiled_view` 按 tile 形状划分数组，load/store 以 tile-space 编号访问数据。边缘 tile 由 tiled view 的 padding 与越界 store 丢弃语义处理，而不是像常见 Triton 写法那样在这段源码中显式传入 `mask`。tile 内每个元素具体映射到哪个线程由 cuTile 编译器决定；`bid` 只选择当前程序实例处理的 tile，并不规定 tile 内的线程分工。与 CUDA SIMT 向量加法相比，这段源码没有显式出现 `threadIdx`、`warp`、`blockDim`、逐线程全局下标计算、逐线程边界分支以及 block 内同步等概念。这些底层机制仍然存在，只是由编译器负责映射和生成。

Editor's Note tile 并非某个 DSL 的发明。NVIDIA 自己把它与 SIMT 并列写进了 Guide 正文，这本身就说明它是一类稳定的编程方式，而不是一次工具选型。

从 SIMT 转向 tile，真正改变的是抽象层的职责划分：线程到数据的映射交给了编译器，tile 的尺寸却仍由用户来定。这条界线不是随意画下的——前者凭形状就可以机械推导，后者必须依赖硬件参数与实测，编译器暂时还无力接手。prob 7.5 的表格会把这一判据推广到四种模型上。

需要明确的是，抽象层的上移并不会抹掉底层的硬件。coalescing、shared memory 的容量、occupancy 这些约束仍然在起作用，只不过不再由你亲手去处理。这也是本作业先用五个模块讲清楚 SIMT，再引入 DSL 的缘故。

7 TileLang 与 Triton

此 Module 用 Triton / TileLang 重写一部分之前用 CUDA SIMT 实现的 kernel，重心在 TileLang。注：Triton 题 (7.1、7.2、7.8) 不需要 GPU 也能完成正确性部分，运行方式见 README.md；TileLang 题都需要 GPU，在集群上跑。

参考资料 TileLang Language Basics^a (建议阅读)；Triton 官方教程 01-vector-add^b；tilelang-puzzles^c。

^ahttps://tilelang.com/programming_guides/language_basics.html

^b<https://triton-lang.org/main/getting-started/tutorials/>

^c<https://github.com/tile-ai/tilelang-puzzles>

模块 7 实验环境与记录口径 (2026-09-11)

实验在 AutoDL 远程容器中运行，工作目录为 assignment01/；7.7 的 benchmark 输出确认 GPU 为 NVIDIA GeForce RTX 3080 Ti。此前测试堆栈显示 Python 3.12；仓库可选依赖固定为 tilelang==0.1.12。本次未保存完整的软件版本查询输出。

远程采用现有 Python 环境运行 python3 -m pytest，避免 uv run 创建独立环境后重新下载大体积依赖；下面保留题目原有的 uv 命令，并在实验记录中写出实际复跑命令。pytest 输出中的秒数是整次测试耗时，不能当作单次 kernel 延迟；性能数据另列于 7.6、7.7。

7.1–7.4、7.6 及 7.7 原始四项测试已有通过记录；7.5 的 cuTile 一列和选做 7.8 仍待完成。以下概念说明按讨论内容和当前代码整理，7.7 为按学员请求提供参考实现后完成的远程验证。

prob 7.1 (FILL-IN)

kernels/vector_add.py

请填空：kernels/vector_add.py，具体要求见代码注释。测试命令：

```
cd assignment01
uv run pytest tests/test_vector_add.py
```

7.1 实现与实验记录

已完成 program 编号、逻辑元素下标及 load/store 的 mask。一个 program 处理一个长度为 BLOCK_SIZE 的逻辑 tile，使用 `tl.program_id(0)` 定位；tile 内元素如何映射到实际线程由编译器安排。因此，BLOCK_SIZE 不是物理线程数，也不保证这些元素同时执行。

grid 向上取整以覆盖全部元素。例如 $N = 2500$ 、BLOCK_SIZE=1024 时启动 3 个 program，最后一个仅有 $2500 - 2 \times 1024 = 452$ 个有效位置。mask 的长度仍为 1024，只有其中 452 个位置为真；读写都需要保护边界，越界 store 可能破坏其他数据。`tl.constexpr` 表示编译期已知的参数，适合决定 `tl.arange` 的逻辑形状，并非仅表示变量不会被修改。

```
python3 -m pytest tests/test_vector_add.py -v
test_exact_multiple PASSED
test_ragged PASSED
test_tiny PASSED
3 passed in 1.52s
```

三个用例分别覆盖长度 4096、10000、3，验证整块、不整齐尾部和极短输入。

prob 7.2 (MODIFY)

kernels/fused_op.py

按文件开头注释要求修改：kernels/fused_op.py。测试命令：

```
cd assignment01
uv run pytest tests/test_fused_op.py
```

改完回答——与 Module 2 里改 CUDA kernel 相比，这次的改动主要集中在 kernel 的什么部分？主体代码为什么一行都不用动？

7.2 实现、作答与实验记录

已实现 $y = \max(ax + b, 0)$ ，保留原有 scale 功能。参数 a, b 使用普通 kernel 参数；本题的逐元素计算不要求它们是编译期常量，避免为每组数值额外生成特化版本。

改动集中在计算表达式、参数列表及调用传参；输入输出形状和数据分工没有改变，所以 program 编号、offsets、mask、load/store 与 grid 框架可以复用。这并不意味着 CUDA 每次改变运算都必须重写索引，同样的逐元素 CUDA 框架也可以复用。融合把乘、加和 ReLU 放进同一个 kernel，避免为每一步单独启动 kernel 并写出中间张量；本题只做了正确性测试，没有量化这部分提速。

```
python3 -m pytest tests/test_fused_op.py -v
test_scale_still_works PASSED
test_fused PASSED
test_fused_other_params PASSED
3 passed in 1.76s
```

测试包括原功能、常用参数以及 $a = -2.0, b = 0.7$ 的另一组参数。

prob 7.3 (FILL-IN)

kernels/tilelang_scale_add.py

请填空: kernels/tilelang_scale_add.py, 具体要求见代码注释。测试命令:

```
cd assignment01
uv sync --extra tilelang
uv run pytest tests/test_tilelang.py -k scale_add
```

7.3 实现与实验记录

已完成二维 grid 与 `T.Parallel(block_M, block_N)` 遍历, 实现 $Y = 2X + 1$ 。x 方向沿列取 $\lceil N/B_N \rceil$ 个 block, y 方向沿行取 $\lceil M/B_M \rceil$ 个 block; 由 tile 起点和内部逻辑坐标得到全局行列号, 并显式判断是否落在输入形状内。

测试形状为 $(M, N) = (123, 77)$, 默认 tile 为 32×32 , 因此 grid 为 (3, 4)。一个 tile 有 1024 个逻辑元素, threads=128 仍可通过每线程处理多个元素完成。

调用过程为: `make_scale_add(M, N)` 生成 `PrimFunc`; `tilelang.compile` 编译该函数, 参数 `out_idx=[1]` 指定第二个参数 Y 为输出; `kernel(x)` 执行并返回结果。`T.Buffer`/`T.Tensor` 是形状和类型声明, 本身不在声明处创建 PyTorch 张量。

```
python3 -m pytest tests/test_tilelang.py -k scale_add -v
test_scale_add PASSED
1 passed, 2 deselected, 2 warnings in 2.74s
```

日志中的 `T.Buffer` 弃用警告建议改用 `T.Tensor`, 本次仍正常编译、运行并通过对拍; `deselected` 表示未选中该文件内的另外两项测试。

prob 7.4 (FILL-IN)

kernels/tilelang_copy2d.py

请填空: kernels/tilelang_copy2d.py, 具体要求见代码注释。测试命令:

```
cd assignment01
uv run pytest tests/test_tilelang.py -k copy2d
```

填完想一想: 2.6 的四个空——行号、列号、边界保护、grid 尺寸——哪些在这里还有对应? 没有对应的那个去哪了?

7.4 实现、作答与实验记录

已完成输入与输出的二维切片: global memory 的 tile 经 `T.copy` 搬到 shared memory, 在 `T.Parallel` 中乘 2, 再复制回对应输出区域。

与 2.6 对照, grid 尺寸仍由用户选择; 行列定位仍体现为 `by*block_M, bx*block_N` 及 tile 的范围。具体元素到线程的映射由编译器安排。源码中不再逐元素写边界 if, 本题的 `T.copy` 根据张量形状和复制区域处理边缘 tile; 用户仍须给出正确的区域与形状, 算法需要的特殊 padding 值也不能一概交给默认边界行为。

```
python3 -m pytest tests/test_tilelang.py -k copy2d -v
test_copy2d PASSED
1 passed, 2 deselected, 8 warnings in 8.18s
```

这一个 pytest 用例内部测试了 (64, 128)、(123, 77)、(1, 500)、(500, 1) 四种形状，包含不整齐边界和单行、单列输入。所贴警告为 T.Buffer 的弃用提示，不是测试失败。

prob 7.5 (CONCEPT)

补全下表，每个空填“用户”或“编译器”（二者都涉及的要写清楚各自的范围）。

谁负责	CUDA SIMT	cuTile	Triton	TileLang
线程到数据的映射	用户	待补	编译器	编译器
边界处理	用户	待补	用户/编译器	用户/编译器
tile / block 尺寸的选择	用户	待补	用户	用户
block 内同步	用户	待补	编译器	编译器

7.5 已讨论部分与适用范围

上表录入已讨论的 Triton、TileLang 部分，cuTile 一列保留待补。

- (a) 线程映射：用户决定每个 program/block 负责哪个逻辑 tile，编译器把 tile 内元素映射到线程。TileLang 中用户显式选择 shared 或 fragment 存储空间，不等于已经指定逐线程的数据分工。
- (b) 边界：Triton 7.1、7.2 的 mask 条件由用户写出，编译器将它降低为受保护的访存。TileLang 7.3 显式写 if，7.4 的复制边界由 T.copy 及编译器处理；7.7 的 $-\infty$ padding 则由用户按 softmax 的归约语义选择。
- (c) 尺寸：用户选择逻辑 tile 尺寸和启动配置；TileLang 的 threads 与逻辑 tile 元素数不必相等。尺寸仍影响寄存器、shared memory 和并发资源。
- (d) 同步：在本节使用的常规 DSL 运算中，编译器安排必要的 block 内同步。用户须正确表达依赖；显式异步拷贝或手工流水仍可能要求用户插入等待。block 内同步也不会自动解决跨 block 的结果合并。

prob 7.6 (FILL-IN)

kernels/tilelang_matmul.py

请填写：kernels/tilelang_matmul.py，具体要求见代码注释。测试命令：

```
cd assignment01
uv run pytest tests/test_tilelang.py -k matmul
```

填完回答——这五个空涉及到了 shared memory、寄存器 tile、流水，而 Triton 版 matmul (Bonus 的 kernels/matmul_triton.py) 并未被显式指定，为什么？

7.6 实现、作答与正确性记录

已完成两块 shared tile、fragment 累加器、沿 K 维分块的流水循环、输入复制及 T.gemm。每个 block 负责 $B_M \times B_N$ 的输出 tile；输入 tile 为 $B_M \times B_K$ 和 $B_K \times B_N$ ，输入使用 fp16，累加器和输出使用 fp32，累加前清零。

TileLang 源码显式表达了数据放在哪种存储空间、如何搬运以及流水深度；当前 Triton 版本主要表达逻辑 tile、t1.load 和 t1.dot，由编译器安排相应的底层存储、布局与调度。不

过 Triton 并非完全不描述累加器: `tl.zeros(..., dtype=tl.float32)` 仍明确指定逻辑形状和精度, tile 尺寸也由用户选择。

```
python3 -m pytest tests/test_tilelang.py -k matmul -v
test_matmul PASSED
1 passed, 2 deselected, 3 warnings in 4.20s
```

测试使用 $(M, N, K) = (256, 256, 128)$ 、 $(B_M, B_N, B_K) = (64, 64, 32)$, 与 `a.float() @ b.float()` 对拍, `atol=rtol=1e-2`。所贴警告仍为 T.Buffer 弃用提示。

7.6 matmul 参数实验 (2026-09-11)

运行 `bench()` 的默认问题规模 $M = N = K = 2048$, 输入 fp16、输出 fp32; `triton.testing.do_bench` 测量编译后的调用。吞吐量按 $2MNK/(t_{\text{ms}} \times 10^{-3})/10^{12}$ 计算。

```
python -c "from kernels.tilelang_matmul import bench; bench()"
```

B_M	B_N	B_K	threads	stages	耗时 (ms)	TFLOPS
64	64	32	128	1	0.287	59.8
64	64	32	128	3	0.290	59.3
128	128	32	128	3	0.342	50.3
128	128	64	256	3	0.374	46.0
128	256	64	256	3	启动失败	

最后一组的日志显示编译完成; 随后调用 kernel, 在设置动态 shared memory 上限时报告:

```
Failed to set the allowed dynamic shared memory size to 147456
```

这是单个 block 请求的动态 shared memory 过大。对 fp16 输入, 按两块输入 tile 及流水缓冲估算:

$$S_{\text{stage}} = 2B_K(B_M + B_N) \text{ bytes}, \quad S_{\text{buffer}} \approx s \cdot S_{\text{stage}}.$$

最后一组每级为 $2 \times 64 \times (128 + 256) = 49152$ bytes, 即 48 KiB; 三级为 144 KiB, 恰好对应报错的 147456 bytes。compute capability 8.6 的每 SM shared memory 为 100 KiB, 每 block 可用上限为 99 KiB, 因此该请求超过硬件上限。^a 前四组按同一模型估算为 8、24、48、96 KiB; 具体占用仍以编译器生成结果为准。

本次有效配置中 (64, 64, 32, 128, 1) 最快。只把 stages 从 1 改为 3, 耗时由 0.287 变为 0.290 ms, 约 1% 的差距不足以单凭一次测量认定稳定优劣。更大的 tile 或更深流水可能增加 shared memory、寄存器压力并减少并发, 不能假定参数越大越快; 当前没有 profiler 数据来确认具体 occupancy 或寄存器溢出。

校验范围: 当前 `bench()` 的 `checked` 标记只让第一组参数与参考结果对拍。后续打印的吞吐量证明完成了计时, 不等于每组都独立通过正确性检查; 最后一组没有有效性能结果。

^aNVIDIA Ampere Tuning Guide, Occupancy 与 Shared Memory: <https://docs.nvidia.com/cuda/ampere-tuning-guide/index.html>

prob 7.7 (FROM-SCRATCH): softmax in TileLang

kernels/tilelang_softmax.py

在 kernels/tilelang_softmax.py 里从零实现行 softmax, 具体要求见文件开头的 docstring。

```
cd assignment01
uv run pytest tests/test_tilelang_softmax.py
```

7.7 参考实现与归约理解

一个 block 负责一整行, 共启动 M 个 block。逻辑行宽补为

$$B_N = \max(32, 2^{\lceil \log_2 N \rceil}), \quad \text{threads} = \min(256, B_N).$$

fragment 存储这一行的逻辑值, 线程协作处理其中的元素; 例如 4096 列可由 256 个线程共同处理, 概念上每线程处理 16 个元素, 不必把一行拆给多个 block。

归约可以理解为: 每线程先合并自己负责的值, 再在线程间合并局部结果。T.reduce_max、T.reduce_sum 描述逻辑维度上的归约, 由编译器安排局部计算、线程间通信和同步。当前一维 fragment 沿 dim=0 归约到长度为 1 的输出 fragment; API 把结果写入输出, 并非返回一个可直接赋值的 Python 数。^a

数值计算顺序为: 有效列加载输入、补齐列置为 $-\infty$; 求行最大值 m_i ; 计算 $e_{ij} = \exp(x_{ij} - m_i)$; 求 $s_i = \sum_j e_{ij}$; 输出 e_{ij}/s_i , 只写回 $j < N$ 的位置。对有限输入, padding 在指数运算后变为 0, 不影响分母。列数超过线程数不是跨 block 拆行的理由; 普通 block 内归约无法自动合并其他 block 的行片段。

wrapper 按形状和 GPU 缓存编译结果, 以 out_idx=[1] 指定输出; 非连续输入先转为连续存储。该实现用于学习和测试, 本次没有进行额外的参数调优。

^aTileLang v0.1.12 归约接口: https://github.com/tile-ai/tilelang/blob/v0.1.12/tilelang/language/reduce_op.py

7.7 正确性实验记录

学员补充的远程输出确认原始四项测试全部通过:

```
python3 -m pytest tests/test_tilelang_softmax.py -v
test_basic PASSED
test_wide_rows PASSED
test_numerical_stability PASSED
test_single_element_rows PASSED
4 passed in 8.71s
```

对应形状为 (33, 127)、(8, 1000)、(4, 256) 和 (5, 1); 数值稳定性用例将随机输入放大 1000 倍并检查输出有限, 全部以 atol=rtol=1e-5 与 torch.softmax 比较。

本地测试文件后来增加了列宽 1025、4095、4096 及全负数不整齐行的用例, 但这份 collected 4 items 日志对应原始测试集, 不能记成扩充后的全部 8 项通过。另有下述 benchmark 对 (4096, 256)、(4096, 1024)、(4096, 4096) 的逐形状对拍通过记录。

7.7 softmax 与 PyTorch 性能对比 (2026-09-11)

GPU 为 NVIDIA GeForce RTX 3080 Ti, 固定 $M = 4096$, 列宽 $N \in \{256, 1024, 4096\}$, 输入为连续 float32 CUDA 张量, `torch.manual_seed(0)`。两种实现使用同一输入, 都通过返回新张量的接口分配输出。

每种形状先调用 `softmax(x)` 触发编译, 再与 `torch.softmax(x, dim=-1)` 对拍并同步; 输入生成、编译和正确性检查位于计时区间之外。分别用以下参数重复测量, 记录 GPU event 耗时的中位数; warmup、rep 均以毫秒为单位:

```
triton.testing.do_bench(
    lambda: softmax(x),
    warmup=100, rep=300, return_mode="median",
)
triton.testing.do_bench(
    lambda: torch.softmax(x, dim=-1),
    warmup=100, rep=300, return_mode="median",
)
```

N	TileLang (ms)	Torch (ms)	加速比	TL (GB/s)	Torch (GB/s)
256	0.0205	0.0133	$0.65\times$	409.6	630.2
1024	0.0461	0.0461	$1.00\times$	728.2	728.2
4096	0.1679	0.1679	$1.00\times$	799.2	799.2

加速比定义为 $t_{\text{Torch}}/t_{\text{TileLang}}$, 大于 1 才表示 TileLang 更快; GB/s 为下述有效带宽。表格保留终端打印精度, 带宽由计时器未舍入的数值计算, 因此用表内四位小数反算可能略有不同。此次没有保存多轮独立实验的波动范围。

7.7 有效带宽、理想下界与结论

对融合的 float32 softmax, 按每元素从显存读一次、向显存写一次估算逻辑数据量:

$$D = 4MN + 4MN = 8MN \text{ bytes}, \quad B_{\text{eff}} = \frac{D}{t_{\text{ms}} \times 10^6} \text{ GB/s}.$$

片上的减法、指数和归约不应逐步重复计入显存字节数。这里的有效带宽按输入输出定义, 区别于 profiler 测得的实际 DRAM 流量。^a

以 RTX 3080 Ti 标称显存带宽 $P = 912 \text{ GB/s}$ 为参考,^b 在数据经过显存、只读写各一次且带宽充分利用的模型下, 理想传输时间为

$$t_{\text{ideal}} = \frac{8MN}{912 \times 10^6} \text{ ms}.$$

N	数据量 (MiB)	理想耗时 (μs)	TL 实测 (μs)	$B_{\text{eff}}/912$
256	8	9.2	20.5	44.9%
1024	32	36.8	46.1	79.8%
4096	128	147.2	167.9	87.6%

$N = 256$ 时, TileLang 延迟约为 Torch 的 1.54 倍, 相差约 $7.2 \mu\text{s}$; 较窄行下, 固定开销、线程利用率和归约安排都可能影响结果, 现有计时不能单独归因。 $N = 1024$ 和 4096 时, 两者在本次显示精度下持平, 不意味着真实耗时逐次完全相同。

行宽增加时, 有效带宽从约 410 提升到 799 GB/s; 在 $N = 4096$ 时达到标称峰值的约 87.6%, 实测 $167.9 \mu\text{s}$ 也较接近理想传输时间 $147.2 \mu\text{s}$ 。宽行结果与显存带宽成为主要限制的解釋相符, 但尚不能证明它是唯一瓶颈。缓存、指数运算、归约、同步和调用开销都会影响结果; 有效带宽与峰值的比值也不是直接测量的 GPU 利用率。

^aNVIDIA CUDA Best Practices Guide, Effective Bandwidth Calculation: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#effective-bandwidth-calculation>

^bRTX 3080 Ti 厂商规格表列出 384-bit、19 Gbps GDDR6X 及 912 GB/s: https://www.gainward.com/main/product/vga/pro/p01138/p01138_datasheet_34161026b20bff75.pdf

prob 7.8 (FROM-SCRATCH · Optional)

kernels/softmax.py

用 Triton 重写 prob 7.7, 此题可以不用 GPU。写完后可以试试对比一下此题与 7.7: 归约、边界处理、按形状编译, 二者各自是由谁处理的 (用户显式处理或者编译器隐式处理)?

测试命令:

```
cd assignment01
uv run pytest tests/test_softmax.py
```

7.8 进度记录

选做, 尚未实现。当前 kernels/softmax.py 仍抛出 NotImplementedError, 没有该题测试通过或性能测量记录; 与 7.7 的归约、边界和按形状编译比较留待完成后补充。

Editor's Note 本模块把前面写过的 kernel 用 TileLang 和 Triton 各重写了一遍。两种写法的区别, 基本都收在 prob 7.5 那张对比表里: 线程到数据的映射归谁管, 边界归谁处理, tile 尺寸归谁选定, 同步又归谁插入。DSL 的价值并不在于代码更短, 而在于它把前两项交给编译器, 同时把后两项——那些必须靠实测才能定下来的旋钮——继续留给你。

抽象当然有代价。Bonus 部分的一组数字提醒你, 表达能力和实现成熟度需要分开评估: 同一段 TileLang 换个版本, 跑出的结果就可能不一样。更关键的是, 一旦编译器接手的那部分出了问题——layout 不合法、精度对不上、边界被悄悄吃掉——你的排查路径最终还是得退回 SIMT 这一层, 去看它究竟生成了什么。DSL 是构筑在 CUDA 之上的一层, 从来不是它的替代品。

8 平台与编译

参考资料 Guide 1.3 全章 (必读)、2.1.1、2.7 (浏览)。

模块 8 前置观察：hello 的 PTX (2026-09-11)

实验继续使用远程 RTX 3080 Ti (CC 8.6)，复用 m0_env/01_hello.cu。该程序启动 4 个 block、每个 block 8 个线程，打印 block 与 thread 编号，并通过错误检查宏检查启动状态及同步结果。

```
make ptx/m0_env/01_hello
sed -n '1,80p' m0_env/01_hello.ptx
```

所贴文件头确认 CUDA 编译工具链为 release 12.8, V12.8.93, PTX 声明如下：

```
.version 8.7
.target sm_52
.address_size 64
```

三项分别表示 PTX 语言版本 8.7、生成代码时假设的目标架构特性、64 位地址。sm_52 是这份 PTX 的目标声明，文件仍是 PTX，并不表示当前 GPU 是 CC 5.2。仓库的 ptx/% 规则未传入 -arch，因此使用 NVCC 12.8 的默认目标；它与普通编译规则中的 ARCH=native 不同。^a

.entry _Z5hellov() 是 hello() 的 kernel 入口，名字经过 C++ 符号修饰。与源码对应的关键指令为：

```
mov.u32 %r1, %tid.x;
mov.u32 %r2, %ctaid.x;
st.local.v2.u32 [%rd2], {%r2, %r1};
```

前两条分别读取 threadIdx.x 和 blockIdx.x；第三条按 block、thread 的顺序准备两个打印参数，后续连同格式字符串传给 vprintf。.reg 声明的是 PTX 虚拟寄存器，不能直接据此计算最终物理寄存器占用；这里用于参数准备的局部存储也不能直接认定为寄存器溢出。

^aNVCC 12.8 默认目标与编译阶段：<https://docs.nvidia.com/cuda/archive/12.8.1/cuda-compiler-driver-nvcc/index.html>

prob 8.1 (CONCEPT)

判断对错，可以顺带补一句理由。

- (a) PTX 是 GPU 直接执行的机器码。
- (b) 只嵌入了 sm_70 SASS 的可执行文件，能在 compute capability 9.0 的卡上运行。
- (c) 一个 fatbin 可以同时携带多个架构的 SASS 和 PTX。
- (d) JIT 编译由驱动在运行时完成。

8.1 作答 (讨论后整理)

- (a) 错。PTX 是虚拟指令集的中间表示，还需要编译成 GPU 可执行的机器码。它可以在构建时经 ptxas 离线编译，也可以保留到运行时再编译。
- (b) 错。本题只含 sm_70 SASS 的程序不能直接在 CC 9.0 GPU 上运行。这里讨论的是机器码的二进制兼容性，不能把 PTX 的前向兼容性套到 SASS 上；对本题涉及的普通目

标，不同 CC 主版本之间没有这种二进制兼容保证。

(c) 对。fatbin 是 GPU 代码容器，可以同时携带面向多个架构的 cubin/SASS 和 PTX；实际包含哪些版本由编译选项决定。

(d) 对。本项首次误答为错，经讨论订正。这里的 JIT 指运行时由 NVIDIA 驱动中的 PTX JIT 编译器生成适合实际 GPU 的机器码；驱动也会缓存结果，因此不是每次启动 kernel 都重新编译。

关键区别是编译发生的阶段：离线编译器和驱动中的运行时编译器都可以完成 PTX 到机器码的转换；“由驱动完成”并不排斥“使用编译器”。^a

^aCUDA Programming Guide 1.3.3–1.3.4, PTX, Cubins and Fatbins, JIT: <https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/cuda-platform.html>

prob 8.2 (EXPERIMENT · Optional)

cuda/m0_env/01_hello.cu

两个编译实验，对象是 Module 0 的 01_hello.cu。

(a) 生成只含 sm_90 SASS 的可执行文件并运行（如果你的卡本身就是 compute capability 9.0，先把 ARCH_HIGH 调成 100 或更高再 make），记录报错信息。

```
cd assignment01/cuda
make sassonly/m0_env/01_hello
./bin/m0_env/01_hello_sassonly
```

8.2(a) 实验记录：只含高架构机器码

远程实际编译命令为：

```
nvcc -O2 -std=c++17 -I. \
-gencode arch=compute_90,code=sm_90 \
-o bin/m0_env/01_hello_sassonly m0_env/01_hello.cu
```

编译成功，Makefile 提示已生成只含 sm_90 机器码的版本。执行后原始报错为：

```
CUDA error cudaErrorNoKernelImageForDevice at m0_env/01_hello.cu:11: no kernel
image is available for execution on the device
```

解释：当前 GPU 为 CC 8.6，而 hello 的设备代码只携带 sm_90 机器码，没有兼容当前 GPU 的机器码或可供后续编译的 PTX，因此运行失败。NVCC 可以为本机没有安装的目标 GPU 生成代码，编译成功本身不证明能在当前设备上执行。报错位置第 11 行是 CUDA_CHECK_KERNEL() 的检查处，并不是设备端 printf 的报错位置。

(b) 生成只含 compute_75 PTX 的版本（CUDA 13 起 compute_70 已被移除，Makefile 默认取 75）并运行。能正常运行吗？PTX 是在什么时候、由谁编译成这块卡的机器码的？

```
cd assignment01/cuda
make ptxonly/m0_env/01_hello
./bin/m0_env/01_hello_ptxonly
```

8.2(b) 实验记录：只含低架构 PTX

远程先执行 `export PATH=/usr/local/cuda/bin:$PATH`，随后成功编译和运行：

```
nvcc -O2 -std=c++17 -I. \
  -gencode arch=compute_75,code=compute_75 \
  -o bin/m0_env/01_hello_ptxonly m0_env/01_hello.cu
./bin/m0_env/01_hello_ptxonly
```

程序正常打印 32 行，每个 block 的线程编号为 0-7。本次输出按 block 0、2、1、3 分组出现；这只是本次打印顺序，不是 CUDA 对 block 调度或执行顺序的保证。首尾输出为：

```
hello from block 0, thread 0
...
hello from block 3, thread 7
```

省略号仅用于此处摘录，终端中实际打印了全部 4×8 行。

解释：hello 的设备代码以 `compute_75` PTX 形式保存在可执行文件中，最终机器码留到运行时由驱动针对实际 GPU 生成。普通 `compute_75` PTX 所要求的硬件能力可由 CC 8.6 GPU 满足，且本次驱动能够处理生成的 PTX，因此程序成功运行。若已有适用的 JIT 缓存，驱动也可以复用缓存；仅凭这次输出无法判断是否发生了新的编译，也没有测量 JIT 耗时。

`compute_75` 指虚拟架构目标，不会强制新工具链输出旧版 PTX 语言；驱动仍须支持实际生成的 PTX 版本。前置观察中的 `.target sm_52` 来自独立的 `ptx/...` 文本生成命令，不能记到本次 `ptxonly/...` 可执行文件上。

8.2 两次实验对照

编译选项中的 code	hello 携带的设备代码	RTX 3080 Ti 上的结果
<code>sm_90</code>	目标架构机器码	无兼容 kernel image，失败
<code>compute_75</code>	PTX 中间代码	正常打印 32 行

前者已经固定了目标机器码；后者保留了运行时针对目标 GPU 完成编译的机会。PTX 的前向兼容性同时受目标硬件能力和驱动支持的 PTX 语言版本约束。

prob 8.3 (CONCEPT · Optional)

请简单说明 Runtime API 与 Driver API 各自的定位。cudaMalloc 属于哪个？

8.3 作答（讨论后整理）

Runtime API 基于 Driver API 实现；cudaMalloc 属于 Runtime API，学员对此判断正确。Runtime API 提供常用操作的较高层封装，简化设备、上下文及 kernel 的使用，适合通常的 CUDA C++ 编程。Driver API 提供更底层、更显式的控制，例如管理上下文、加载代码模块和启动 kernel；应用可以直接使用 Driver API，也可以按其互操作规则与 Runtime API 配合使用。^a

^aCUDA Programming Guide 1.3.2.1, CUDA Runtime API and CUDA Driver API: <https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/cuda-platform.html#cuda-runtime-api-and-cuda-driver-api>

Editor's Note Module 0 里那个“能跑就行”的 hello，现在被拆开重新看过：源码经 nvcc 分成 host 和 device 两路，device 端按编译选项将 PTX、SASS 或二者打包进 fatbin；运行时 driver 再从中挑选合适的版本，或者对 PTX 做 JIT。

这一模块的实用意义在交付环节。“编译通过”和“能在目标卡上跑起来”是两回事，后者是部署中最常撞上的墙。当你把一个 GPU 程序交付给别人，至少要说清楚：它支持哪些 compute capability，fatbin 里包含了哪些架构的 SASS，有没有 PTX 兜底，以及首次运行的 JIT 开销落在哪里。写 kernel 与发布软件是两种视角，而这里正是二者的交界。

Bonus (EXPERIMENT · Optional): matmul

调参并比较不同 matmul 实现的性能差距。

- (a) Naive CUDA: `cuda/bonus/matmul.cu`, 用 `-DBS` 取 8、16、32 各跑一遍, 记录实测数据。

```
cd assignment01/cuda
make run/bonus/matmul
nvcc -O2 -std=c++17 -I. -arch=native -DBS=8 -o bin/bonus/matmul_bs8 bonus/matmul.cu
&& ./bin/bonus/matmul_bs8
```

- (b) Tiled Triton: `kernels/matmul_triton.py` (多组 tile 配置 + cuBLAS 对照), 记录实测数据。

```
cd assignment01
uv run python -c "from kernels.matmul_triton import bench; bench()"
```

- (c) TileLang: `kernels/tilelang_matmul.py` (即 prob 7.6 补全的成品), 调 `bench()` 的 `block_M / block_N / block_K / num_stages`, 记录实测数据。

```
cd assignment01
uv sync --extra tilelang
uv run python -c "from kernels.tilelang_matmul import bench; bench()"
```

试分析性能差距的原因, 特别是: TileLang 的控制粒度比 Triton 更细, 为什么在这组测试里反而略慢?

注: A100 实测参考——naive CUDA 约 2.4 TFLOPS, Triton 约 125 TFLOPS, TileLang 约 118 TFLOPS, cuBLAS 约 173 TFLOPS, 而 A100 fp16 tensor core 的标称上限是 312 TFLOPS。Triton 与 TileLang 的数字只是各自 `bench()` 里那几组配置中的最优, 不代表工具的性能上限; tilelang 本身也还在快速迭代 (本仓库固定在 0.1.12), 数字会随版本变化。

Editor's Note 回顾这份作业, 主线可以看作三次视角的切换: 单线程到 warp (Module 1–3)、计算到数据搬运 (Module 4–5)、线程到 tile (Module 6–7)。走完这一遍, 你写下的 kernel 应当能保证正确, 且不至于慢得离谱。但距离“快”字仍然有一段差距, Bonus 里那组数字恰好把它量化了出来——naive 实现和 cuBLAS 之间, 隔着 tensor core、asynchronous copy、software pipelining 以及 profiler-driven tuning, 这些内容会在后续的 session 中展开。